上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

2026年春季学期

编译原理

Principles of Compilers

第二章：一个简单的语法制导翻译器

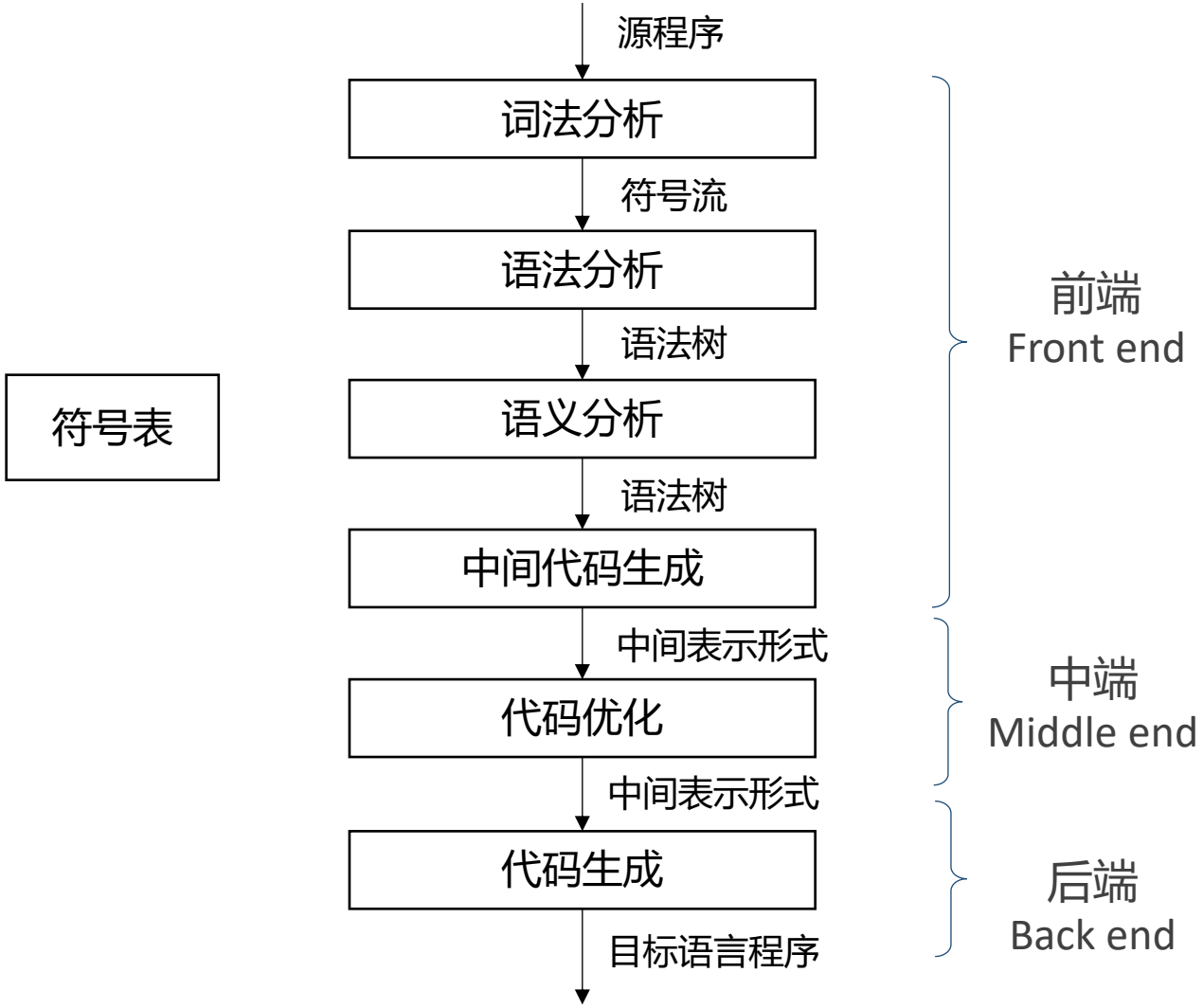
赵杰茹

上海交通大学计算机学院

回顾：编译器的基本结构

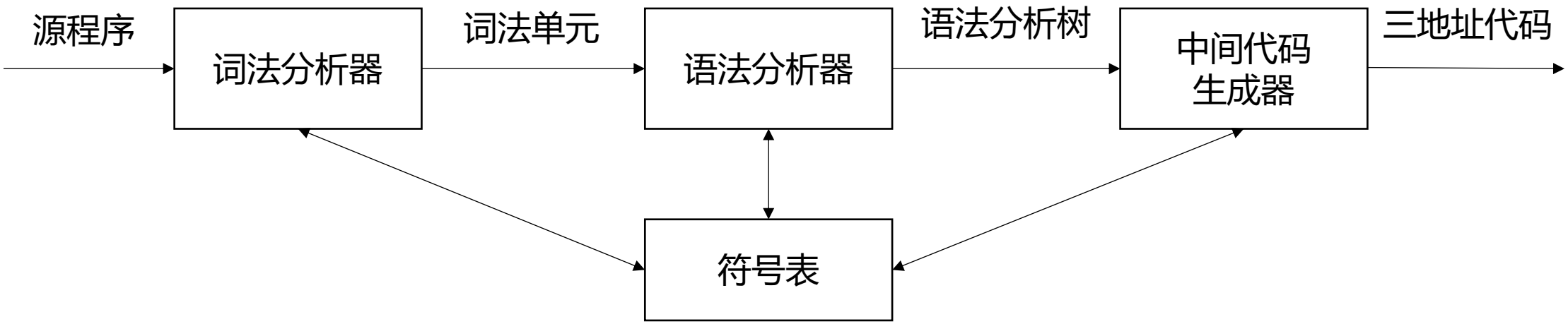
编译器的基本结构：

1. 词法分析
2. 语法分析
3. 语义分析
4. 中间代码生成
5. 代码优化
6. 代码生成



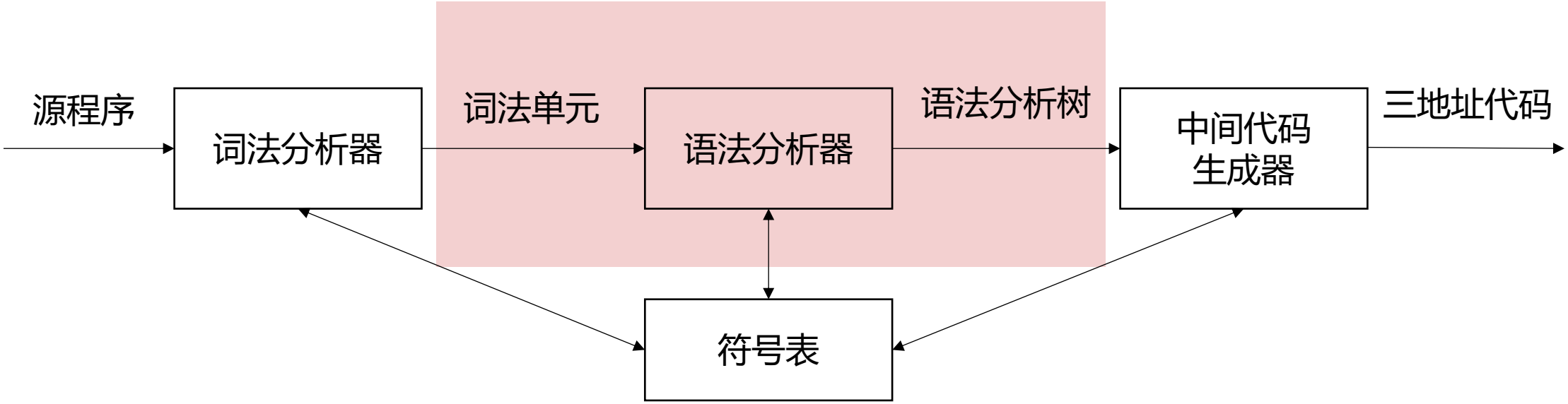
回顾：编译器前端

- 编译器前端是一个分析(analysis)过程
  - 将源程序分解成多个组成要素，加上语法结构，构建出源程序的中间表示(IR: intermediate representation)



回顾：编译器前端

- 分析阶段是围绕着编译语言的“**语法**”展开的
  - 语法分析器：分析输入的词法单元序列，生成语法分析树
  - 我们将通过一个例子，理解语法分析的基本理论与实践方法



内容纲要

- 语法定义
  - 上下文无关文法
  - 基本概念
- 推导
  - 推导概念与实例
  - 语法分析树
  - 二义性
- 翻译
  - 语法制导定义
  - 语法制导翻译方案

内容纲要

- 语法定义
  - 上下文无关文法
  - 基本概念
- 推导
  - 推导概念与实例
  - 语法分析树
  - 二义性
- 翻译
  - 语法制导定义
  - 语法制导翻译方案

语法定义

- 上下文无关文法，简称“文法”，是一种广泛使用的、描述程序设计语言语法的表示方法
- 以 if-else 语句为例：

if (expression) statement **else** statement

其构造规则可以表示为：

stmt -> **if** (*expr*) *stmt* **else** *stmt*

注：箭头“->”读作“可以具有如下形式”，这样的规则称为产生式(production)。**if**，**else**和括号等词法元素称为终结符号(terminal)，*expr* 和 *stmt* 等变量表示终结符号的序列，称之为非终结符号(nonterminal)

上下文无关文法

- 上下文无关文法（context-free grammar）包含四个元素：
  - 一个**终结符号**集合：该文法所定义语言的基本符号的集合
  - 一个**非终结符号**集合：每个非终结符号表示一个终结符号串
  - 一个**产生式**集合
    - 每个产生式包括一个称为**产生式头或左部**的非终结符号，一个箭头，和一个称为**产生式体或右部**的由终结符号及非终结符号组成的序列
    - 一种书写方式：表示某个构造

      stmt -> **if** (*expr*) *stmt* **else** *stmt*
  - 一个**开始符号**：指定一个非终结符号为开始符号

上下文无关文法

- 描述文法时，需列出该文法的产生式，并首先列出以开始符号为头部的产生式。
- 同时我们假设：
  - 数字、符号（如<、<=）和黑体字符串（如**while**）都是终结符号
  - 斜体字符串表示非终结符号
  - 所有非斜体的名字或者符号可以看作是终结符号
- 为表示方便，将具有相同头部的产生式合并，表示成一个产生式
  - 其头部为所有产生式共有的非终结符号
  - 产生式体为所有产生式体的集合，用“|”分割

例1：描述一个表达式的文法

- 描述由数字、加号和减号组成的表达式
  - 比如 9-5+2、3-1、7等，即“用加号和减号分隔的数字序列”
  - 此文法的产生式为：

$$list \rightarrow list + digit$$
$$list \rightarrow list - digit$$
$$list \rightarrow digit$$
$$digit \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

  - 归纳总结
    - 该文法的终结符号包括：+ - 0 1 2 3 4 5 6 7 8 9
    - 该文法的非终结符号为 *list* 和 *digit*
    - 该文法的开始符号为*list*（其产生式首先被提出）

例1：描述一个表达式的文法

- 描述由数字、加号和减号组成的表达式
  - 比如 9-5+2、3-1、7等，即“用加号和减号分隔的数字序列”
  - 此文法的**产生式**为：

$$list \rightarrow list + digit$$
$$list \rightarrow list - digit$$
$$list \rightarrow digit$$
$$digit \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

  - 以非终结符号*list*为头部的三个产生式也可以合并为：

$$list \rightarrow list + digit \mid list - digit \mid digit$$
$$digit \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

思考：函数调用语句的文法

- 如何描述函数调用语句的文法?

思考：函数调用语句的文法

- 如何描述**函数调用**语句的文法?
  - 参数列表可以有多个参数，也可以为空
  - 我们可以构造出如下的产生式:

$$call \rightarrow \textbf{id}\ (\ optparams\ )$$
$$optparams \rightarrow params \mid \epsilon$$
$$params \rightarrow params\ ,\ param \mid param$$

  - 说明:
    - 含有零个终结符号的串称为空串，记为 ϵ
    - 函数参数可以是多种形式，根据具体语言来定义 *param* 的产生式

内容纲要

- 语法定义
  - 上下文无关文法
  - 基本概念
- 推导
  - 推导概念与实例
  - 语法分析树
  - 二义性
- 翻译
  - 语法制导定义
  - 语法制导翻译方案

推导

- 什么是推导?
  - 语法分析是一个**推导**过程：接受一个终结符号串作为输入，找出从文法的开始符号推导出这个串的方法；如果不能推导出来，则报告语法错误
  - 根据文法推导符号串时，首先**从开始符号出发**，不断将某个非终结符号**替换**为该非终结符号的某个产生式的体，每替换一次，被称为一次**推导**

- 什么是语言?
  - 可以从文法的开始符号推导得到的所有终结符号串的集合，称为该文法定义的**语言**（language）

例2：根据表达式文法推导符号串

- 例1得到的表达式文法：

$$list \rightarrow list + digit \quad (1)$$
$$list \rightarrow list - digit \quad (2)$$
$$list \rightarrow digit \quad (3)$$
$$digit \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \quad (4)$$

  - 我们可以按照如下方法分析出 9-5+2 是不是一个*list*
    1) 因为9是*digit*，根据产生式(3)可知 9 是一个*list*;
    2) 因为5是*digit*，9是*list*，根据产生式(2)可知 9-5 也是一个*list*;
    3) 因为2是*digit*，9-5是*list*，根据产生式(1)可知 9-5+2 是一个*list*。

上述推导的过程可以表示为：

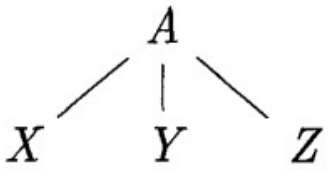
list => list + digit => list - digit + digit => digit - digit + digit => 9 - digit + digit => 9 - 5 + digit => 9-5+2

  - 该文法定义的语言：由加减号分隔的数字列表的集合

语法分析树

- 语法分析树
  - 以**树**的形式**展现**了从文法的开始符号推导出符号串的过程
  - 对于给定文法，相应的语法分析树具有以下性质：
    - 树的**根结点**的标号为文法的开始符号
    - 每个**叶子结点**的标号为一个终结符号或者 ϵ
    - 每个**内部结点**的标号为一个非终结符号
    - 如果非终结符号A是某个内部结点的标号，$X1,X2,\cdots,Xn$ 是该结点从左到右排列的所有子结点的标号，则必然存在产生式 $A\rightarrow X1X2\cdots Xn$，其中，$X1,X2,\cdots,Xn$ 可以是终结符号或非终结符号

产生式: *A -> XYZ*　　语法分析树:

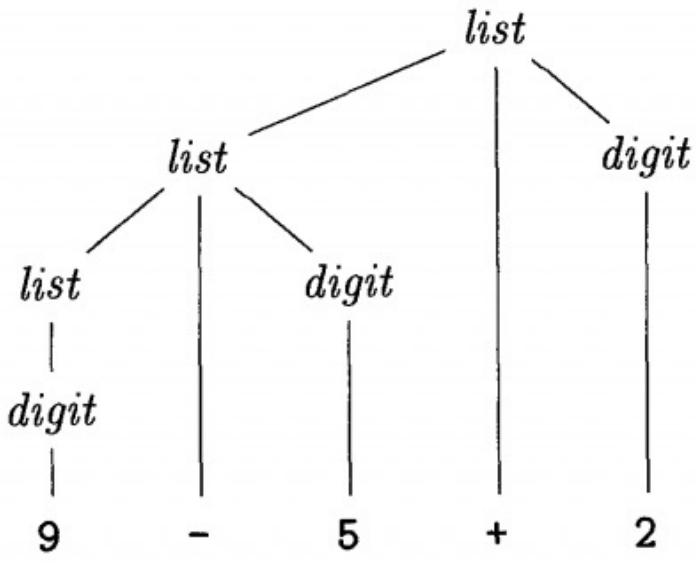


例3：用树形图演示推导过程

- 表达式：9 - 5 + 2
- 推导过程：

list => list + digit => list - digit + digit => digit - digit + digit => 9 - digit + digit => 9 - 5 + digit => 9-5+2

- 语法分析树：
  - 每个结点的标号都是一个文法符号
  - 内部结点和它的子结点对应于一个产生式
    - 内部结点对应产生式的头
    - 子结点对应产生式的体
  - 叶子结点从左向右构成了树的结果，即推导出的符号串



语法分析树

- 语法分析树
  - 以**树**的形式**展现**了从文法的开始符号推导出符号串的过程

- 使用语法分析树的概念，我们可以定义：
  - 一个文法的**语言**是指任何能够由某棵语法分析树生成的符号串的集合
  - 而**语法分析**是为一个给定的终结符号串构建一颗语法分析树的过程

二义性

- 回顾：对比人类语言
  - “Jack said Jerry left his assignment at home”
    - 这里的“his”代指谁呢？Jack还是Jerry？
- 二义性（ambiguous）
  - 针对**同一个终结符号串**，一个文法可能有**多棵语法分析树**能够生成此符号串
- 如何判断文法是否有二义性？
  - 找到一个终结符号串，说明其是两棵以上语法分析树可以生成的结果

例4：判断二义性

- 表达式文法：

list -> *list* + *digit* | *list* – *digit* | *digit*

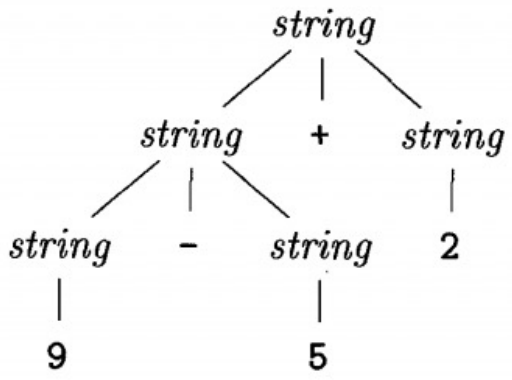
digit -> 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

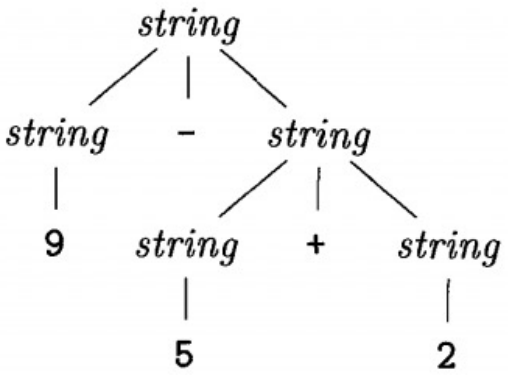
- 如果不区分*digit*和*list*，该文法可以改写成：

string -> *string* + *string* | *string* – *string* | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

- 尝试分析表达式 9 – 5 + 2，会得到两棵语法分析树

(9-5)+2





9-(5+2)
产生错误结果

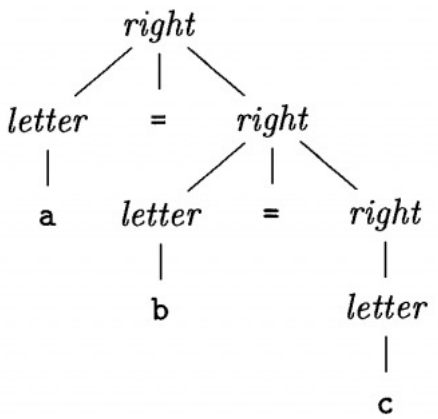
运算符的结合性

- 通常来讲，9+5+2 等价于 (9+5)+2，这体现了运算符的**结合性**
- 当一个运算分量左右两侧都有运算符时，我们需要制定规则来决定哪个运算符被应用于哪个运算分量
  - 左结合：该运算分量属于左边的运算符，比如加、减、乘、除等运算
    - 9+5+2 等价于 (9+5)+2
  - 右结合：该运算分量属于右边的运算符，比如赋值操作“=”
    - a=b=c 等价于 a=(b=c)
- 运算符的结合性需要通过文法定义来实现

连续赋值语句的文法：

$$
\begin{array}{lcl}
right & \rightarrow & letter = right \mid letter \\
letter & \rightarrow & \mathtt{a} \mid \mathtt{b} \mid \cdots \mid \mathtt{z}
\end{array}
$$

a=b=c的语法分析树：



运算符的优先级

- 通常来讲，9+5*2 等价于 9+(5*2)，这体现了运算符的**优先级**
  - 即 * 比 + 具有更高的优先级
- 运算符的优先级也需要通过文法定义来实现

非终结符号 *expr* 的产生式，处理+ - 操作

$$
\begin{array}{lcl}
expr & \rightarrow & expr + term \\
 & | & expr - term \\
 & | & term
\end{array}
$$

非终结符号 *term* 的产生式，处理* /操作

$$
\begin{array}{lcl}
term & \rightarrow & term * factor \\
 & | & term \ / \ factor \\
 & | & factor
\end{array}
$$

非终结符号 *factor* 生成基本单元

$$
factor \rightarrow \mathbf{digit} \mid (\ expr \ )
$$

实现优先级的文法：

$$
\begin{array}{lcl}
expr & \rightarrow & expr + term \mid expr - term \mid term \\
term & \rightarrow & term * factor \mid term \ / \ factor \mid factor \\
factor & \rightarrow & \mathbf{digit} \mid (\ expr \ )
\end{array}
$$

问题：如何根据该文法构建出9+5*2的语法分析树？

内容纲要

- 语法定义
  - 上下文无关文法
  - 基本概念
- 推导
  - 推导概念与实例
  - 语法分析树
  - 二义性
- 翻译
  - 语法制导定义
  - 语法制导翻译方案

翻译

- 翻译实例：**将中缀表达式翻译成后缀表达式**
- 一个表达式E的后缀表示（postfix notation）定义如下：
  - 如果E是一个变量或者常量，则E的后缀表示是E本身;
  - 如果E是形如 E_1 **op** E_2 的表达式，其中 **op** 是一个二元操作符，则E的后缀表示是 E_1' E_2' **op**，这里 E_1' 和 E_2' 分别是 E_1 和 E_2 的后缀表示;
  - 如果E是形如(E_1)的表达式，则E_1的后缀表示是E的后缀表示。
- 例如：
  - (9-5)+2的后缀表示是95-2+
  - 9-(5+2)的后缀表示是952+-

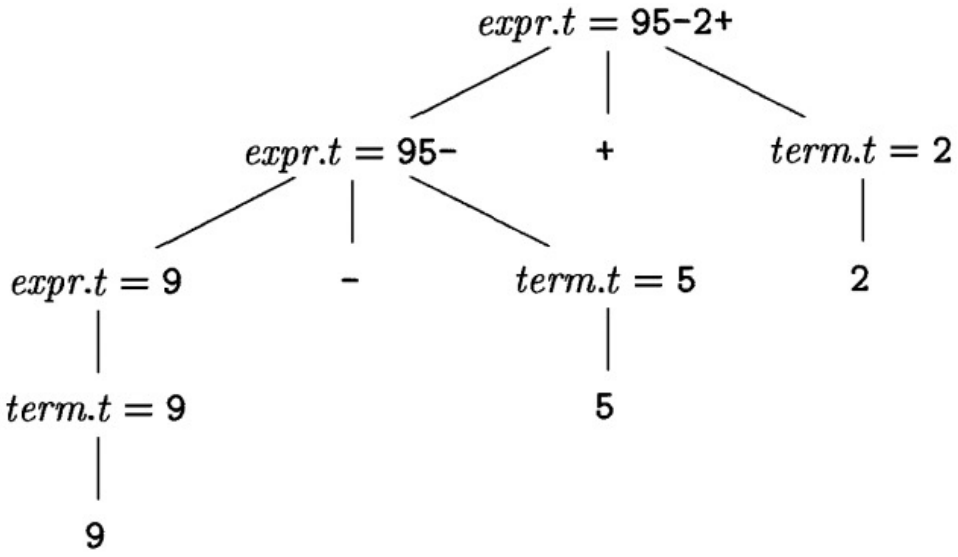
回顾

- 词法单元被表示为<token-name, attribute-value>
  - attribute- value 指向符号表相应条目（存有附加信息）
- 终结符号是文法符号，不具有词法单元的附加信息
- 为了实现翻译，我们需要将**属性**添加回来
  - 表达式的数据类型
  - 生成代码中的指令数目
  - 生成指令的位置
  - 等等

语法制导定义

- 语法制导定义（syntax-directed definition）
  - 将每个**文法符号**和一个**属性**集合（比如数值及类型）相关联
  - 将每个**产生式**和一组**语义规则**相关联，用于计算文法符号相应的属性值

- 以表达式 9-5+2 为例
  - 为每个文法符号添加属性 t
  - 生成注释（annotated）语法分析树
  - 根据语义规则计算各结点属性值



语法制导定义

- 将中缀表示翻译成后缀表示的语法制导定义

产生式	语义规则
$expr \rightarrow expr_1 + term$	$expr.t = expr_1.t \mid\mid term.t \mid\mid '+'$
$expr \rightarrow expr_1 - term$	$expr.t = expr_1.t \mid\mid term.t \mid\mid '-'$
$expr \rightarrow term$	$expr.t = term.t$
$term \rightarrow 0$	$term.t = '0'$
$term \rightarrow 1$	$term.t = '1'$
$\cdots$	$\cdots$
$term \rightarrow 9$	$term.t = '9'$

其中，$expr_1$这里的数字下标仅仅被用于将产生式体中的$expr$实例和产生式头区别开来，

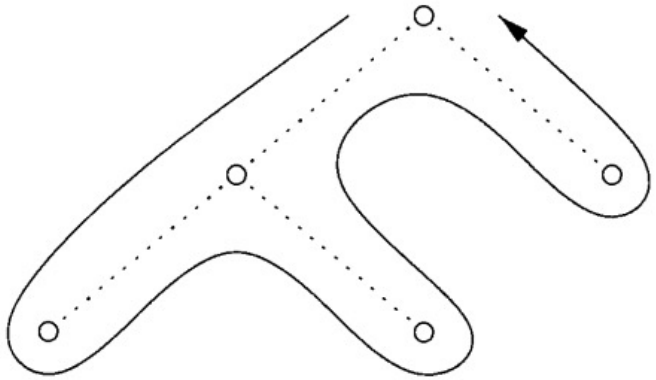
||表示字符串的连接运算符

遍历语法树

- 树的遍历：
  - 从根结点开始，以某种顺序访问树的每一个节点。
  - 采用方法：**深度优先遍历（depth-first）**
    - 从根结点开始，递归地按照任意顺序访问各结点的子结点，不一定从左往右
    - 尽可能地访问一个结点的尚未被访问的子结点，因此总是尽可能快地访问离根结点最远的结点（最深的结点）

```
procedure visit(node N) {
        for ( each child C of N, from left to right ) {
                visit(C);
        }
        evaluate semantic rules at node N;
}
```

一棵树的深度优先遍历



遍历图示

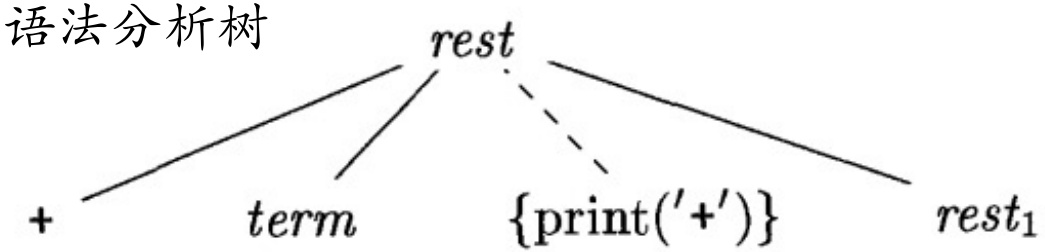
语法制导翻译方案

- **语法制导定义**是将字符串作为**属性值**附加到结点上，再根据语义规则遍历树得到翻译结果
- **语法制导翻译方案**是一种在文法产生式中附加一些**程序片段**来描述翻译结果的表示方法
  - 嵌入到产生式体中的程序片段，被称为**语义动作**，用花括号表示
  - 和语法制导定义类似，只是显式指定了语义规则的计算顺序

产生式

rest -> + *term* { *print*('+') } *rest1*

语法分析树



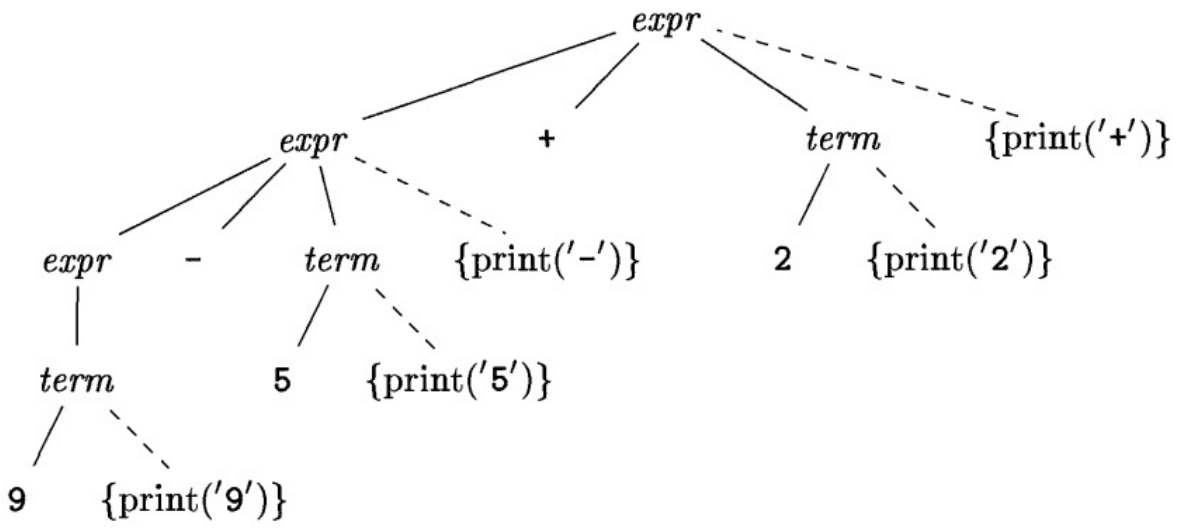
语法制导翻译方案

- 以表达式 9-5+2 为例
  - 添加额外的叶子结点（打印操作），通过虚线连接内部结点
  - 对整棵树进行从左到右的深度优先遍历，访问叶子结点时执行打印语句
  - 内嵌的语义动作将输入表达式翻译成后缀表达式

翻译方案的文法

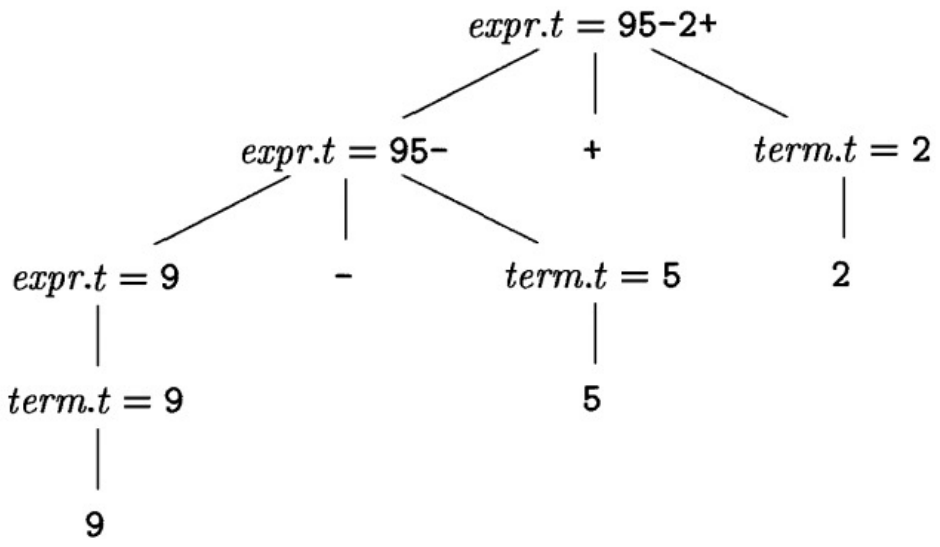
expr	→	$expr_1$ + *term*	{print('+')}
expr	→	$expr_1$ - *term*	{print('-')}
expr	→	*term*	
term	→	0	{print('0')}
term	→	1	{print('1')}
		...	
term	→	9	{print('9')}

包含语义动作的语法树

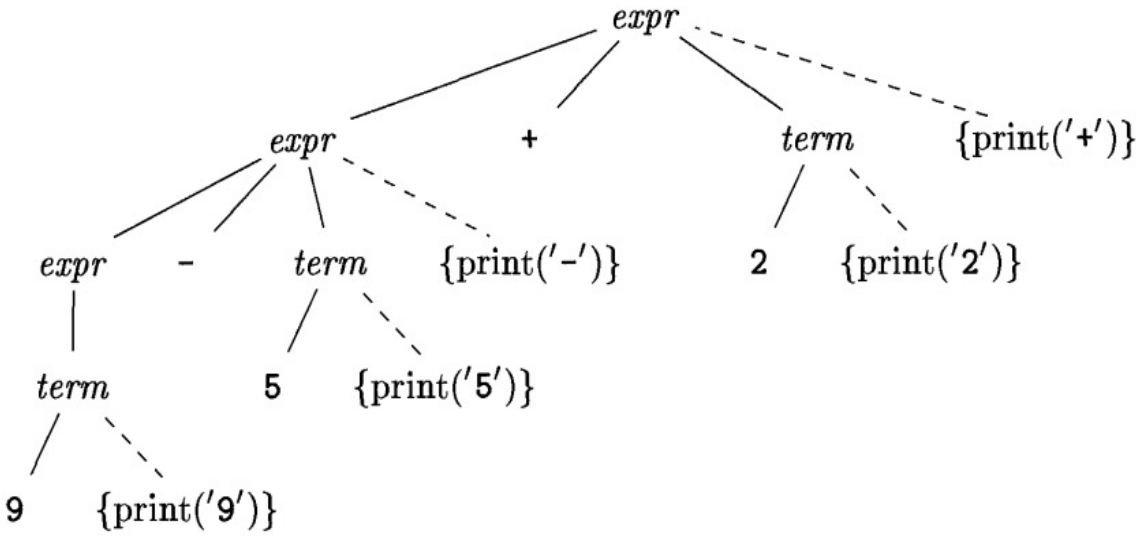


小结

- 语法制导定义将字符串作为属性附加到语法分析树的各结点上
- 语法制导翻译方案通过语义动作，将翻译结果递增地打印出来，不需要附加空间来存放子表达式的翻译结果



语法分析树中的各结点的属性值



实现翻译的语义动作